

Managed Agents > Advanced orchestration

Multiagent orchestration

 Copy page 

Coordinate multiple agents within a single session.

Multiagent orchestration lets one agent coordinate with others to complete complex work. Agents can act in parallel with their own isolated context, which helps improve output quality and can also improve time to completion.

Not sure a multiagent setup fits your problem? See [when to use multiagent systems \(and when not to\)](#).

 Managed Agents API requests require the `managed-agents-2026-04-01` beta header, except memory store endpoints, which use `agent-memory-2026-07-22` instead. The SDK sets the correct beta header automatically. See [Beta headers](#).

How it works

All agents share the same sandbox, filesystem, and [vault credentials](#), but each agent runs in its own **session thread**, a context-isolated event stream with its own conversation history. The coordinator reports activity in the **primary thread** (which is the same as the session-level [event stream](#)); additional threads are spawned at runtime when the coordinator delegates work.

Threads are persistent: the coordinator can send a follow-up to an agent it called earlier, and that agent retains everything from its previous turns.

Each agent uses its own configuration: model, system prompt, tools, MCP servers, and skills. Session-level [agent configuration overrides](#) are the exception; they apply to the coordinator and its `self` copies. Tools, MCP servers, and context are not shared.

What to delegate

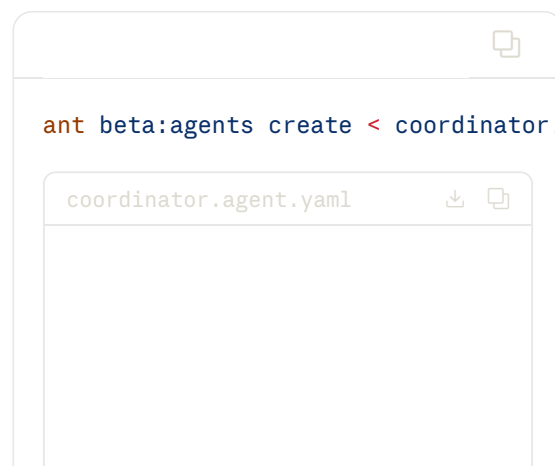
Multiagent coordination is best suited for complex tasks that either require work across a variety of surfaces, or where multiple well-scoped tasks contribute to an overall goal.

Patterns that work well:

- **Parallelization:** Fan out independent subtasks simultaneously (searching multiple sources, analyzing separate files) and have the coordinator synthesize the results.
- **Specialization:** Route to agents with domain-focused system prompts and tools, such as a security agent or a documentation agent, rather than loading a single agent with every capability.
- **Escalation:** Consult a more capable agent or model for a subset of complex subtasks.

Configure the coordinator

When [defining your agent](#), set `multiagent` to declare the roster of agents the coordinator can delegate to:



```

name: Engineering Lead
model: claude-opus-5
system: You coordinate engineering
tools:
  - type: agent_toolset_20260401
multiagent:
  type: coordinator
  agents:
    - type: agent
      id: $REVIEWER_AGENT_ID # re
    - type: agent
      id: $TEST_WRITER_AGENT_ID #

```

`multiagent.agents` can accept any of the following:

- `{"type": "agent", "id": agent.id}` references a previously created `agent` by ID. If no `version` is specified, the reference is pinned to the latest version of that agent at the time the coordinator is created.
- `{"type": "agent", "id": agent.id, "version": agent.version}` pins a specific agent version.
- `{"type": "self"}` allows the coordinator to spawn copies of itself. If the session was created with [agent configuration overrides](#), those overrides also apply to these copies; roster entries referenced by ID are unaffected.
- `{"type": "advisor", "model": "<model id>"}` gives the session's primary thread an advisor it can consult mid-turn. At most one advisor entry per roster. See [Give the session an advisor](#).

The coordinator's configuration, including its `multiagent.agents` roster, is snapshotted when the coordinator is created or updated. Referenced agents stay pinned to the versions resolved at that time and do not automatically pick up later updates to their definitions. To delegate to a newer version of a referenced agent,

[update the coordinator](#) so its roster references that version.

The coordinator can only delegate to one level of agents; referencing an agent that has its own `multiagent.agents` roster fails the create or update request with a validation error. A maximum of 20 unique agents can be listed in `multiagent.agents`, but the coordinator can call multiple copies of each agent.

When agents pin an [inference geography](#) (`model.inference_geo` in the [agent definition](#)), the coordinator's pin and every roster member's pin must either all be set to the same value or all be unset. A mismatched roster is rejected with a 400 validation error, both when the agent is saved and when a [session-create override](#) changes any of the pins.

Give the session an advisor

An advisor entry in `multiagent.agents` gives the session's primary thread an **advisor**: a model it can consult mid-turn for strategic guidance, such as planning an approach, getting unstuck, or reviewing work before finishing. The entry has exactly two fields, `type` and `model`:

cURL

```
curl -fsS https://api.anthropic.com/v
-H "x-api-key: $ANTHROPIC_API_KEY"
-H "anthropic-version: 2023-06-01"
-H "anthropic-beta: managed-agents-
-H "content-type: application/json"
-d '{
  "name": "Backend engineer",
  "model": "claude-sonnet-5",
  "system": "You implement backend
  "multiagent": {
    "type": "coordinator",
    "agents": [
      {"type": "advisor", "model":
    ]
  }
}'
```

A roster can contain at most one advisor entry, alongside any of the other roster forms. The entry occupies the reserved roster name `anthropic.advisor`: a roster that lists both an advisor entry and a member literally named `anthropic.advisor` is rejected with a 400 validation error. In responses, the advisor entry is echoed last in the roster regardless of the position it was submitted in.

The advisor model must meet a minimum capability bar, and the agent's own model must not be more capable than its advisor; models of equal capability can pair. An invalid pairing is rejected with a 400 validation error when the agent is saved. Valid pairings follow the advisor tool's [model compatibility](#) table.

The advisor is also available as a [server tool on the Messages API](#). The Managed Agents surface differs in configuration and delivery: the roster entry has no `max_uses`, `max_tokens`, or `caching` fields, and advice arrives through thread events rather than `advisor_tool_result` blocks.

How consultations work

Each consultation runs as a platform-spawned thread named `anthropic.advisor` that terminates itself when the consultation completes, and the advice is delivered to the primary thread as an `agent.thread_message_received` event. A consultation emits the standard thread events, identified by the reserved name `anthropic.advisor` (the thread lifecycle events carry it as `agent_name`, and the advice delivery carries it as `from_agent_name`), typically in this order:

1. `session.thread_created`
2. `session.thread_status_running`
3. `agent.thread_message_received` (the advice)
4. `session.thread_status_idle` (`stop_reason: end_turn`)

5. `session.thread_status_terminated`

No `agent.tool_use` events are emitted for a consultation, and no `agent.thread_message_sent` event appears on the session's event stream, because the consultation input is composed by the platform rather than sent by the agent. If you list the advisor thread's own events, the advice also appears there as an `agent.thread_message_sent` event. The advice delivery (event 3) is not guaranteed to arrive before the advisor thread's idle and terminated events, so don't treat those as a signal that the advice has already been delivered.

Whether your client can read the advice is the advisor model's policy, and it mirrors the [result variants](#) split on the Messages API advisor tool. Advisor models that return plaintext results there deliver the advice as readable text content here; advisor models that return redacted results there deliver a `[{"type": "redacted"}]` placeholder as the message content on every client surface, while the agent itself still reads the full advice server-side. In the preceding example, Claude Opus 5 is a redacted-result advisor, so your client sees the placeholder while the agent reads the full advice; choose Claude Opus 4.8 as the advisor instead if you want the advice readable on the event stream. Advisor thinking is never surfaced. Clients cannot send `redacted` blocks themselves; an event containing one is rejected with a 400 validation error.

A failed or interrupted consultation never fails the agent's turn: the agent continues after a generic notice that the consultation failed. A session-level `user.interrupt` during a consultation terminates the advisor thread with no advice delivered; a `user.interrupt` with the advisor thread's `session_thread_id` abandons only that consultation.

Advisor threads

The advisor is not a roster agent: it is invisible to the coordinator's `list_agents` tool, it cannot be messaged with `send_to_agent`, and only the session's primary thread can consult it. Roster agents cannot.

Advisor threads are exempt from the concurrent-thread limit. They appear in the session's [thread list](#) with `agent` set to the advisor form exactly as configured (`{"type": "advisor", "model": ...}`) and `parent_thread_id` set to the primary thread.

Prompt caching on the advisor's side is automatic; there is nothing to configure. Consultations are billed at the advisor model's rates, and their tokens appear in the advisor thread's usage and in the session's usage totals.

Removing the advisor

To remove the advisor, [update the agent](#) with a roster that no longer includes the advisor entry. If the advisor is the roster's only entry, clear the roster entirely by setting `"multiagent": null`.

Create the session

Create a session referencing the coordinator. The coordinator delegates to the agents in its roster as needed.



```
session = client.beta.sessions.create
    agent=coordinator.id,
    environment_id=environment.id,
)
```

Connect agents to MCP servers

MCP servers are agent-scoped (each agent definition declares its own servers and

tools), while vault credentials are session-scoped (`vault_ids` passed at session

Q Search

⌘ K

Subscribe to webhooks

Define outcomes

Authenticate with vaults

Manage agent context

Access GitHub

Attach and download files

Build persistent memory >

Advanced orchestration

Multiagent orchestration

Scheduled deployments

Reference

Managed Agents reference

Working with files

Files API

PDF support

Images and vision



Béatrice
Admin · Béatrice's l...



- To authenticate MCP servers, include a vault credential for every MCP server used across all agents.
- To limit an agent's access, declare only the servers it needs in its agent definition.

[Agent configuration overrides](#) at session creation can replace the coordinator's MCP servers and those of its `self` copies.

⌂

```
research_agent = client.beta.agents.create(
    name="researcher",
    model="claude-haiku-4-5",
    mcp_servers=[
        {"type": "url", "name": "github"},
    ],
    tools=[{"type": "mcp_toolset", "mcp_server": "github"}],
)

coordinator = client.beta.agents.create(
    name="coordinator",
    model="claude-opus-5",
    tools=[{"type": "agent_toolset_20250901", "agent": "coordinator"}],
    multiagent={
        "type": "coordinator",
        "agents": [{"type": "agent", "name": "researcher"}],
    },
)

session = client.beta.sessions.create(
    agent=coordinator.id,
    environment_id=environment.id,
    vault_ids=[vault.id],
)

print(session.id)
```

In this example, only the researcher declares the GitHub MCP server, so the coordinator does not have access. The session's `vault_ids` supply the GitHub credential to the researcher's thread.

Ask Docs



💡 If an agent's MCP calls fail to authenticate after you declare the server, confirm the credential's `mcp_server_url` refers to the same server as the agent's `mcp_servers[].url`. Both URLs are normalized before matching (scheme and host lowercased, default ports and trailing slashes stripped), so differences in host casing, a default port, or a trailing slash don't prevent a match; a different path, subdomain, or non-default port does.

Threads

The **session-level event stream**

(`/v1/sessions/{session_id}/events/stream`) is considered the **primary thread**, containing a condensed view of all activity across all threads. You don't see the full activity from subagents, but you do see the start and end of their work, and blocking events such as tool permission requests.

Session threads are where you drill into a specific agent's activity.

The session `status` is an aggregation of all agent activity; if at least one thread is `running`, then the overall session status is `running` as well.

A [session budget](#) is a single shared cap across all of a session's threads. As the cap is reached, threads pause independently, and each thread's cost is priced at the thread's own served model.

📝 A maximum of 25 concurrent threads is supported. The coordinator can call multiple copies of a single agent in the roster, creating multiple threads associated with one `agent`. [Advisor](#) consultation threads are exempt from this limit.

List all threads associated with a session as follows:



```
for thread in client.beta.sessions.th
    print(f"[{thread.agent.name}] {th
```

The full list includes the primary thread.
`parent_thread_id` is null for the primary thread.

Primary thread events

These events surface multiagent activity on the primary thread at

`/v1/sessions/{session_id}/events/stream`. Message-direction events are named relative to the thread whose stream they appear on:

`agent.thread_message_received` means a message arrived on this thread from another thread, and

`agent.thread_message_sent` means this thread sent one. The task the coordinator delegates, for example, arrives on the child's own stream as an

`agent.thread_message_received` event.

Type	
<code>session.thread_created</code>	[...]
<code>session.thread_status_running</code>	[...]
<code>session.thread_status_idle</code>	[...]
<code>session.thread_status_terminated</code>	[...]
<code>agent.thread_message_received</code>	[...]
<code>agent.thread_message_sent</code>	[...]

Advisor consultations emit these same thread events under the reserved name `anthropic.advisor` (as `agent_name` on the thread lifecycle events and `from_agent_name` on the advice delivery); see [Give the session an advisor](#) for the sequence.

Session thread events

Critical events are proxied to the primary thread. However, you might still want to investigate a specific agent's reasoning and tool calls. To do so, stream or list the events from the associated session thread.

Each session thread has its own event stream at

`/v1/sessions/{session_id}/threads/{thread_id}/stream`, and it accepts the same `event_deltas[]` parameter as the session-level stream, so you can preview a subagent's text as the model generates it. A connection previews only the thread it's reading: a child thread's previews never appear on the session-level stream, so to watch a subagent live, open its own thread stream. See [Preview session thread events](#) for opting in, accumulating, and reconciling previews.

```
with client.beta.sessions.threads.event_stream(
    thread_id,
    session_id=session.id,
) as stream:
    for event in stream:
        match event.type:
            case "agent.message":
                for block in event.content:
                    if block.type == "text":
                        print(block.text)
            case "session.thread_status_change":
                break
```

Tool permissions and custom tools

If a subagent needs something from your client, such as [permission](#) to run an `always_ask` tool, or the [result of a custom tool](#), the event is cross-posted to the **primary thread** with `session_thread_id` identifying the originating session thread.

```
{
  "type": "session.thread_status_idle",
  "id": "sevt_01ABC...",
  "session_thread_id": "sth_01DEF...",
  "agent_name": "code-reviewer",
  "stop_reason": {
    "type": "requires_action",
    "event_ids": ["sevt_01XYZ..."]
  }
}
```

Post `user.tool_confirmation` (with `tool_use_id`) or `user.custom_tool_result` (with `custom_tool_use_id`); the server routes the response to the correct thread automatically.

The following example extends the [tool confirmation handler](#) to route replies. The same pattern applies to `user.custom_tool_result`.

```
for event_id in stop.event_ids:
    client.beta.sessions.events.send(
        session.id,
        events=[
            {
                "type": "user.tool_co
                "tool_use_id": event_
                "result": "allow",
            }
        ],
    )
```

Was this page helpful?



Claude Platform Docs



Solutions, Compliance, and security, Help and security, Blog, Courses, Availability, AI Anthropic, Agents, Career, Code, Economics, Status, modernization, Future, Support, Coding, Research, Connectors, Discord, Customer, News, Customer support, Responsible, Financial, Scaling, Engineering, services, Policy, at, Government, Anthropic, Privacy, Higher, Events, policy, education, compliance, Powered, Responsible, K-12, Transparency, disclosure, Claude, policy, teachers, Service, Terms, Life, partners, of, sciences, Startups, Service: program, Partners, Terms, Claude, of, on, service:, AWS, Consumer, Claude, Usage, on, policy, Google, Cloud